MeisterTask Integration

1. Overview

1.1 Description

MeisterTask Integration connects **MeisterTask** (project and task management platform) with **Newo Platform**.

It enables:

- Creating tasks with names, notes, and due dates via conversation
- Looking up tasks by name and reporting their status
- Updating task details (due date, notes, section)
- Marking tasks as completed
- Adding comments to existing tasks
- Listing all open tasks in a project

This integration is designed for **teams and businesses** that need to manage tasks through conversational AI without switching to a separate project management interface.

1.2 Use Cases

- When a caller reports an issue -> the AI agent creates a task in MeisterTask
- When a caller asks about a task status -> the agent looks it up and reports details
- When a caller needs to reschedule work -> the agent updates the task due date
- When a caller confirms work is done -> the agent marks the task as completed
- When a caller provides follow-up info -> the agent adds a comment to the task
- When a caller needs an overview -> the agent lists all open tasks

1.3 Supported Features

Feature	Supported	Notes
Create task	Yes	With name, notes, due date
Look up task by name	Yes	Full task list sent to LLM for matching
Update task details	Yes	Due date, notes, section; searches by name if no ID
Complete task	Yes	Checks status before completing (prevents double-complete)
Add comment	Yes	Requires task ID from prior lookup
List open tasks	Yes	Filtered by status=open
DevMode (LLM emulation)	Yes	Emulates API responses without real MeisterTask
Multi-project support	No	Uses single default project; auto-resolves if not set
Task assignment	No	Schema extracted but not routed to API
Webhook notifications	No	No inbound webhook support

2. Requirements

Before installation:

- Active **MeisterTask** account (Business or Pro plan recommended)
- **Personal Access Token** from MeisterTask API settings
- **Project ID** and **Section ID** for the target project (found in URL)

3. How to Setup

3.1 Step 1 - Generate Credentials in MeisterTask

1. Log in to your **MeisterTask** account
2. Navigate to **Account Settings -> API -> Personal Access Tokens**
3. Click **Generate New Token**
4. Copy the token and store securely (it is shown only once)
5. Note your **Project ID** from the project URL: `https://www.meistertask.com/app/project/{PROJECT_ID}/...`
6. Note your **Section ID** by opening the project and checking network requests, or use the API: `GET /projects/{PROJECT_ID}/sections`

3.2 Step 2 - Connect in Platform

1. Open **Newo -> Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`meistertask_api_key`	Yes	MeisterTask personal access token (Bearer token)
`meistertask_base_url`	No	API base URL (default: `https://www.meistertask.com/api`)
`meistertask_default_project_id`	Yes	Project ID where tasks are created and listed
`meistertask_default_section_id`	Yes	Section (column) ID for new tasks
`meistertask_mode`	No	`Production` (default) or `DevMode` for LLM emulation
`meistertask_setup_scenarios`	No	`True` (default) to auto-add scenarios to canvas

3. Click **Save + Publish All**
4. On publish, the InitFlow automatically:
   - Creates the `meistertask_connector` HTTP connector
   - Registers 6 custom tools with ConvoAgent
   - Adds task management scenarios to the canvas (if enabled)

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow (typically from ConvoAgent detecting a user intent)
- An **Action** is the API operation performed in MeisterTask

Available Triggers

Trigger	Event	Description
Create task requested	`meistertask_create_task_tool`	User asks to create a new task
Lookup task requested	`meistertask_lookup_task_tool`	User asks to find or check a task
Update task requested	`meistertask_update_task_tool`	User asks to change task details
Complete task requested	`meistertask_complete_task_tool`	User asks to mark a task as done
Add comment requested	`meistertask_add_comment_tool`	User asks to add a comment to a task
List tasks requested	`meistertask_list_tasks_tool`	User asks to see all open tasks
Integration published	`project_publish_finish`	Triggers setup and tool registration
Canvas built	`gm_workflow_builder_canvas_built`	Triggers scenario injection into canvas

Available Actions

- **Create Task** - `POST /sections/{section_id}/tasks` with name, notes, due date
- **List Tasks** - `GET /projects/{project_id}/tasks?status=open` filtered by open status
- **Look Up Task** - `GET /projects/{project_id}/tasks?status=open` then match by name
- **Update Task** - `PUT /tasks/{task_id}` with changed fields (notes, due, section)
- **Complete Task** - `GET /tasks/{task_id}` to check status, then `PUT /tasks/{task_id}` with `{status: 2}`
- **Add Comment** - `POST /tasks/{task_id}/comments` with comment text

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant CA as ConvoAgent
    participant TM as TaskManagementFlow
    participant API as ApiFlow
    participant NW as NetworkFlow
    participant MT as MeisterTask API

    Note over TM: Setup (on publish)
    CA->>TM: project_publish_finish
    TM->>CA: Register 6 custom tools

    Note over U,MT: Create Task
    U->>CA: "Create a task Review Q2 budget"
    CA->>CA: Confirm details with user
```

```
    U->>CA: "Yes"
    CA->>TM: meistertask_create_task_tool (memory)
    TM->>TM: Gen() extract task_name, notes, due_date
    TM->>API: meistertask_create_task_request
    API->>NW: meistertask_execute_request
    NW->>MT: POST /sections/{id}/tasks
    MT-->>NW: 200 {id, name}
    NW->>API: meistertask_result_success
    API->>TM: meistertask_create_task_success
    TM->>CA: urgent_message "Task created"
    CA->>U: "The task has been created"

    Note over U,MT: Look Up Task
    U->>CA: "Look up task Review Q2"
    CA->>TM: meistertask_lookup_task_tool (memory)
    TM->>TM: Gen() extract search_term
    TM->>API: meistertask_lookup_task_request
    API->>NW: GET /projects/{id}/tasks?status=open
    NW->>MT: GET tasks
    MT-->>NW: 200 [task list]
    NW->>API: meistertask_result_success
    API->>TM: meistertask_lookup_task_success (full list)
    TM->>CA: urgent_message with task details
    CA->>U: "Task is Open, due April 5"

    Note over U,MT: Complete Task
    U->>CA: "Mark it as completed"
    CA->>TM: meistertask_complete_task_tool (memory)
    TM->>TM: Gen() extract task_id
    TM->>API: meistertask_complete_task_request
    API->>NW: GET /tasks/{id} (check status)
    NW->>MT: GET task
    MT-->>NW: 200 {status: 1}
    NW->>API: status is Open
    API->>NW: PUT /tasks/{id} {status: 2}
    NW->>MT: PUT task
    MT-->>NW: 200
    NW->>API: meistertask_result_success
    API->>TM: meistertask_complete_task_success
    TM->>CA: urgent_message "Task completed"
    CA->>U: "Task has been marked as completed"
```

InitFlow

```
flowchart TD
    E["project_publish_finish"] --> CHECK{"registry_item_idn\n==
meistertask_integration?"}
    CHECK -->|"No"| RET["Return()"]
    CHECK -->|"Yes"| INIT["_initSkill()\nCreate connector\nSet attributes\nCreate
persona"]
```

```
    INIT --> TOOLS["_setupToolsSkill()\nRegister 6 custom tools\nvia
generalmanageragent_update_newo_tools"]
```

TaskManagementFlow

```
flowchart TD
    TOOL["Custom tool event\n(from ConvoAgent)"] --> EXTRACT["Entry skill\nGen()
extract params\nfrom conversation memory"]
    EXTRACT --> VALID{"Required params\npresent?"}
    VALID -->|"No"| ASK["urgent_message\nAsk user for missing info"]
    VALID -->|"Yes"| REQ["SendSystemEvent\nmeistertask_*_request"]
    REQ --> API["ApiFlow processes request"]
    API --> SUCCESS["meistertask_*_success"] -->
RESULT["TaskResultSuccessSkill\nStore in persona\nSend urgent_message"]
    API --> ERROR["meistertask_task_error"] --> ERRMSG["TaskResultErrorSkill\nSend
urgent_message\nwith error details"]
```

ApiFlow - Create Task

```
flowchart TD
    E["meistertask_create_task_request"] --> PROJ{"default_project_id\nconfigured?"}
    PROJ -->|"No"| GETP["GET /projects"]
    GETP --> GOTP["_createTask_gotProjectsSkill\nSelect first project"]
    GOTP --> SEC{"default_section_id\nconfigured?"}
    PROJ -->|"Yes"| SEC
    SEC -->|"No"| GETS["GET /projects/{id}/sections"]
    GETS --> GOTS["_createTask_gotSectionsSkill\nSelect first section"]
    GOTS --> CREATE["POST /sections/{id}/tasks\n{name, notes?, due?}"]
    SEC -->|"Yes"| CREATE
    CREATE --> OK{"HTTP 200?"}
    OK -->|"Yes"| SUCC["_createTaskSuccessSkill\nEmit
meistertask_create_task_success"]
    OK -->|"No"| ERR["_emitTaskErrorSkill\nEmit meistertask_task_error"]
```

ApiFlow - Lookup Task

```
flowchart TD
    E["meistertask_lookup_task_request"] --> PROJ{"default_project_id\nconfigured?"}
    PROJ -->|"No"| GETP["GET /projects"]
    GETP --> GOTP["_lookupTask_gotProjectsSkill\nSelect first project"]
    GOTP --> TASKS["GET /projects/{id}/tasks\n?status=open"]
    PROJ -->|"Yes"| TASKS
    TASKS --> LIST["_lookupTask_gotTasksSkill\nFormat task summaries\nSend full list
to ConvoAgent"]
    LIST --> SUCC["meistertask_lookup_task_success\nLLM matches task by name"]
```

ApiFlow - Update Task

```
flowchart TD
    E["meistertask_update_task_request"] --> ID{"task_id\nprovided?"}
    ID -->|"No, but task_name"| SEARCH["GET /projects/{id}/tasks\n?status=open"]
    SEARCH --> FOUND["_updateTask_gotTasksSkill\nSend task list for\nuser confirmation"]
    ID -->|"Yes"| BODY{"Update fields\npresent?"}
    BODY -->|"No"| ERR["_emitTaskErrorSkill\nNo fields to update"]
    BODY -->|"Yes"| PUT["PUT /tasks/{task_id}\n{notes?, due?, section_id?}"]
    PUT --> OK{"HTTP 200?"}
    OK -->|"Yes"| SUCC["_updateTaskSuccessSkill\nEmit meistertask_update_task_success"]
    OK -->|"No"| ERR2["_emitTaskErrorSkill"]
    ID -->|"Neither"| ERR3["_emitTaskErrorSkill\ntask_id or task_name required"]
```

ApiFlow - Complete Task

```
flowchart TD
    E["meistertask_complete_task_request"] --> GET["GET /tasks/{task_id}\nCheck current status"]
    GET --> CHECK{"Status?"}
    CHECK -->|"status == 2\n(Completed)"| ALREADY["Already completed\nEmit success"]
    CHECK -->|"status == 8 or 16\n(Trashed/Archived)"| ERR["Cannot complete\nEmit error"]
    CHECK -->|"status == 1\n(Open)"| PUT["PUT /tasks/{task_id}\n{status: 2}"]
    PUT --> SUCC["_completeTaskSuccessSkill\nEmit meistertask_complete_task_success"]
```

ApiFlow - Add Comment

```
flowchart TD
    E["meistertask_add_comment_request"] --> VALID{"task_id and\ncomment_text present?"}
    VALID -->|"No"| ERR["_emitTaskErrorSkill"]
    VALID -->|"Yes"| POST["POST /tasks/{task_id}/comments\n{text: comment_text}"]
    POST --> OK{"HTTP 200?"}
    OK -->|"Yes"| SUCC["_addCommentSuccessSkill\nEmit meistertask_add_comment_success"]
    OK -->|"No"| ERR2["_emitTaskErrorSkill"]
```

ApiFlow - List Tasks

```
flowchart TD
    E["meistertask_list_tasks_request"] --> PROJ{"project_id\nconfigured?"}
    PROJ -->|"No"| GETP["GET /projects"]
    GETP --> GOTP["_listTasks_gotProjectsSkill\nSelect first project"]
    GOTP --> TASKS["GET /projects/{id}/tasks\n?status=open"]
    PROJ -->|"Yes"| TASKS
```

```
    TASKS --> SUCC["_listTasksSuccessSkill\nFormat and
emit\nmeistertask_list_tasks_success"]
```

NetworkFlow

```
flowchart TD
    E["meistertask_execute_request"] --> MODE{"meistertask_mode?"}
    MODE -->|"Production"| EXEC["_executeRequestSkill\nBuild HTTP request\nBearer
auth\nSendCommand"]
    MODE -->|"DevMode"| EMU["_emulateRequestSkill\nGen() simulates\nAPI response"]
    EXEC --> HTTP["HTTP Connector\nmeistertask_connector"]
    HTTP --> R200{"Status 200/201?"}
    R200 -->|"Yes"| RS["ResultSuccessSkill\nGetAct() recover params\nEmit
meistertask_result_success"]
    R200 -->|"No"| RE["ResultSuccessSkill\nEmit meistertask_result_error"]
    HTTP --> RERR["ResultErrorSkill\nEmit meistertask_result_error"]
    EMU --> RS2["Emit meistertask_result_success\nwith emulated response"]
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. For testing without a real MeisterTask account, set `meistertask_mode` to `DevMode` or use a mock server.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify that 6 custom tools appear in `project_attributes_settings_newo_tools`
2. **Create Task**: Say "Create a task called Test task with a note test note" -> verify task appears in MeisterTask
3. **Look Up Task**: Say "Look up the task Test task" -> verify agent reports name, status, due date
4. **Update Task**: Say "Update the task Test task, change due date to next Friday" -> verify change in MeisterTask
5. **Complete Task**: Say "Mark Test task as completed" -> verify status changes to Completed
6. **Add Comment**: Say "Add a comment to Test task: This is done" -> verify comment appears
7. **List Tasks**: Say "List all open tasks" -> verify all open tasks are shown

If no action occurs:

- Ensure `meistertask_api_key` is set and valid
- Verify `meistertask_default_project_id` and `meistertask_default_section_id` are set
- Confirm all flows are published after configuration changes
- Check that custom tools are registered in `project_attributes_settings_newo_tools`
- Verify `meistertask_mode` is `Production` (not `DevMode`) for real API calls
- Check server logs for error events (`meistertask_task_error`)

> *Note: This integration performs real operations in MeisterTask. Changes made through the agent are reflected in the live system.*

5. FAQ

Q: Can I use the integration without a MeisterTask account? A: Yes. Set `meistertask_mode` to `DevMode` and the integration will emulate API responses using LLM. No real tasks are created.

Q: How does the agent find tasks by name? A: The integration fetches all open tasks from the configured project and sends the full list to ConvoAgent. The LLM matches the user's request to the correct task from the list.

Q: What happens if I don't configure project_id and section_id? A: The integration will auto-resolve by fetching the first available project and its first section. This adds extra API calls and may select the wrong project if you have multiple.

Q: Can the agent update a task without knowing its ID? A: Yes. If the user refers to a task by name, the integration searches open tasks and asks the user to confirm the match before updating.

Q: What task statuses does the integration recognize? A: Open (1), Completed (2), Trashed (8), Archived (16). The complete action only works on Open tasks. Trashed and Archived tasks cannot be completed.

Q: Are completed tasks included in lookups and listings? A: No. All task queries use `?status=open` to filter only active tasks.

Q: Can I connect multiple MeisterTask projects? A: Each integration instance supports one default project. For multiple projects, create separate integration instances.